

LAPORAN
(TUGAS SISTEM BOOKING HOTEL)



Dosen pengampu :
Triawan Adi Cahyanto M.Kom

Disusun Oleh:
Nama : Yulian Adinda Dwi M.S.S
Nim : 2410651015
Kelas : A – Teknik Informatika
Tanggal : 14 November 2025

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

DAFTAR ISI

DAFTAR ISI.....	i
A. SOAL PEMROGRAMAN.....	ii
1. Analisis Masalah.....	1
2. Implementasi.....	1
3. Testing & Output.....	6
4. Analisis Hasil.....	7
B. KESIMPULAN	
C. REFERENSI	

SOAL PEMOGRAMAN

1. ANALISIS MASALAH

A. Pemahaman Soal

Program ini mensimulasikan sistem booking hotel dengan 5 kamar yang dapat dipesan oleh 10 pelanggan secara bersamaan (menggunakan 10 thread). Setiap pelanggan akan mencoba memesan kamar maksimal 3 kali. Jika kamar penuh, pelanggan menunggu 2 detik sebelum mencoba kembali. Jika berhasil, pelanggan “menginap” selama 5 detik lalu check out.

B. Identifikasi Masalah Sinkronisasi

Masalah utama pada program ini adalah terjadinya race condition saat beberapa thread mencoba mengakses sumber daya bersama (jumlah kamar tersedia dan operasi cetak).

Untuk mengatasi hal ini, digunakan:

- Semaphore → membatasi jumlah kamar yang dapat diakses.
- Mutex → mencegah output terminal saling tumpang tindih antar thread.

C. Desain Solusi

- Gunakan semaphore untuk menjaga jumlah kamar yang tersedia (maksimal 5).
- Gunakan mutex untuk mengatur tampilan print agar rapi.
- Setiap pelanggan dijalankan sebagai thread terpisah.

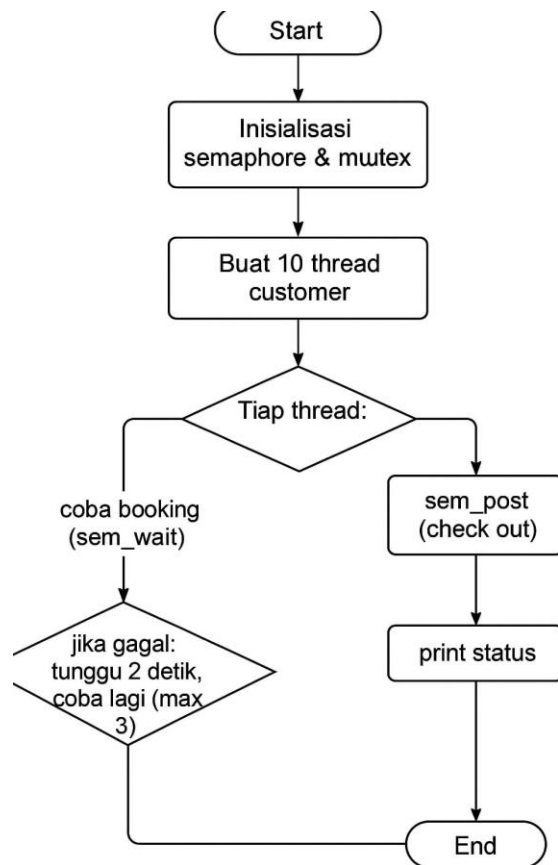
2. IMPLEMENTASI

A. Penjelasan Algoritma

1. Inisialisasi semaphore dengan nilai 5 (jumlah kamar).
2. Setiap thread pelanggan mencoba memesan kamar:
 - a. Jika kamar tersedia (`sem_wait()` berhasil), tampilkan pesan berhasil.

- b. Jika tidak, tunggu 2 detik dan coba lagi (maksimal 3 kali).
3. Jika berhasil, pelanggan “menginap” selama 5 detik, lalu melakukan sem_post() untuk melepaskan kamar.
4. Gunakan mutex untuk mengunci bagian printf.

B. Flowchart :



C. Hasil Percobaan (Source Code Program)

Buat file baru

nano booking_hotel.c

lalu ketik code berikut :

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define JUMLAH_KAMAR 5
#define JUMLAH_CUSTOMER 10

sem_t kamar_tersedia;
pthread_mutex_t print_lock;

int kamar[JUMLAH_KAMAR] = {0};

void* customer_booking(void* arg) {
    int id = *(int*)arg;
    int coba = 0;

    while (coba < 3) {
        coba++;
        sleep(rand() % 3 + 1);

        pthread_mutex_lock(&print_lock);
        printf(" Customer %d mencoba booking (percobaan ke-%d)...\n", id, coba);
        pthread_mutex_unlock(&print_lock);

        if (sem_trywait(&kamar_tersedia) == 0) {
            pthread_mutex_lock(&print_lock);

            int kamar_dipesan = -1;
            for (int i = 0; i < JUMLAH_KAMAR; i++) {
                if (kamar[i] == 0) {
                    kamar[i] = 1;
                    kamar_dipesan = i;
                    break;
                }
            }

```

```

    }

    int sisa_kamar;
    sem_getvalue(&kamar_tersedia, &sisa_kamar);
    printf(" Customer %d berhasil booking kamar #%d (tersisa %d kamar)\n",
           id, kamar_dipesan + 1, sisa_kamar);

    pthread_mutex_unlock(&print_lock);

    sleep(5);

    pthread_mutex_lock(&print_lock);
    kamar[kamar_dipesan] = 0;
    sem_post(&kamar_tersedia);
    sem_getvalue(&kamar_tersedia, &sisa_kamar);
    printf("customer %d check-out dari kamar #%d (tersisa %d kamar)\n",
           id, kamar_dipesan + 1, sisa_kamar);
    pthread_mutex_unlock(&print_lock);

    pthread_exit(NULL);
} else {
    pthread_mutex_lock(&print_lock);
    printf("customer %d gagal booking (kamar penuh), menunggu 2 detik...\n", id);
    pthread_mutex_unlock(&print_lock);
    sleep(2);
}
}

pthread_mutex_lock(&print_lock);
printf("customer %d menyerah setelah 3 kali gagal booking.\n", id);
pthread_mutex_unlock(&print_lock);

pthread_exit(NULL);
}

int main() {
    srand(time(NULL));

```

```

pthread_t tid[JUMLAH_CUSTOMER];
int id[JUMLAH_CUSTOMER];

sem_init(&kamar_tersedia, 0, JUMLAH_KAMAR);
pthread_mutex_init(&print_lock, NULL);

printf("\n=== SISTEM BOOKING HOTEL ===\n");
printf("Total Kamar: %d\n\n", JUMLAH_KAMAR);

for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
    id[i] = i + 1;
    pthread_create(&tid[i], NULL, customer_booking, &id[i]);
}

for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
    pthread_join(tid[i], NULL);
}

sem_destroy(&kamar_tersedia);
pthread_mutex_destroy(&print_lock);

printf("\n=== Semua customer selesai ===\n");
return 0;
}

```

Jika sudah lalu jalankan menggunakan code berikut :

```
gcc booking_hotel.c -o booking_hotel -lpthread
./booking_hotel
```

Alasan memilih mekanisme tersebut, karena semaphore cocok untuk mengontrol jumlah sumber daya terbatas (kamar hotel), sedangkan mutex digunakan agar hasil print antar-thread tidak tumpang tindih.

3. TESTING & OUTPUT

```
=== SISTEM BOOKING HOTEL ===
Total Kamar: 5

customer 10 mencoba booking (percobaan ke-1)...
customer 10 berhasil booking kamar #1 (tersisa 4 kamar)
customer 6 mencoba booking (percobaan ke-1)...
customer 6 berhasil booking kamar #2 (tersisa 3 kamar)
customer 7 mencoba booking (percobaan ke-1)...
customer 7 berhasil booking kamar #3 (tersisa 2 kamar)
customer 4 mencoba booking (percobaan ke-1)...
customer 4 berhasil booking kamar #4 (tersisa 1 kamar)
customer 5 mencoba booking (percobaan ke-1)...
customer 5 berhasil booking kamar #5 (tersisa 0 kamar)
customer 3 mencoba booking (percobaan ke-1)...
customer 3 gagal booking (kamar penuh), menunggu 2 detik...
customer 9 mencoba booking (percobaan ke-1)...
customer 9 gagal booking (kamar penuh), menunggu 2 detik...
customer 2 mencoba booking (percobaan ke-1)...
customer 2 gagal booking (kamar penuh), menunggu 2 detik...
customer 8 mencoba booking (percobaan ke-1)...
customer 8 gagal booking (kamar penuh), menunggu 2 detik...
customer 1 mencoba booking (percobaan ke-1)...
customer 1 gagal booking (kamar penuh), menunggu 2 detik...
customer 3 mencoba booking (percobaan ke-2)...
customer 3 gagal booking (kamar penuh), menunggu 2 detik...
customer 10 check-out dari kamar #1 (tersisa 1 kamar)
customer 9 mencoba booking (percobaan ke-2)...
customer 9 berhasil booking kamar #1 (tersisa 0 kamar)
customer 6 check-out dari kamar #2 (tersisa 1 kamar)
customer 1 mencoba booking (percobaan ke-2)...
customer 1 berhasil booking kamar #2 (tersisa 0 kamar)
customer 7 check-out dari kamar #3 (tersisa 1 kamar)
customer 4 check-out dari kamar #4 (tersisa 2 kamar)
customer 2 mencoba booking (percobaan ke-2)...
customer 2 berhasil booking kamar #3 (tersisa 1 kamar)
customer 5 check-out dari kamar #5 (tersisa 2 kamar)
customer 8 mencoba booking (percobaan ke-2)...
customer 8 berhasil booking kamar #4 (tersisa 1 kamar)
customer 3 mencoba booking (percobaan ke-3)...
customer 3 berhasil booking kamar #5 (tersisa 0 kamar)
customer 9 check-out dari kamar #1 (tersisa 1 kamar)
customer 1 check-out dari kamar #2 (tersisa 2 kamar)
customer 2 check-out dari kamar #3 (tersisa 3 kamar)
customer 3 gagal booking (kamar penuh), menunggu 2 detik...
customer 10 check-out dari kamar #1 (tersisa 1 kamar)
customer 9 mencoba booking (percobaan ke-2)...
customer 9 berhasil booking kamar #1 (tersisa 0 kamar)
customer 6 check-out dari kamar #2 (tersisa 1 kamar)
customer 1 mencoba booking (percobaan ke-2)...
customer 1 berhasil booking kamar #2 (tersisa 0 kamar)
customer 7 check-out dari kamar #3 (tersisa 1 kamar)
customer 4 check-out dari kamar #4 (tersisa 2 kamar)
customer 2 mencoba booking (percobaan ke-2)...
customer 2 berhasil booking kamar #3 (tersisa 1 kamar)
customer 5 check-out dari kamar #5 (tersisa 2 kamar)
customer 8 mencoba booking (percobaan ke-2)...
customer 8 berhasil booking kamar #4 (tersisa 1 kamar)
customer 3 mencoba booking (percobaan ke-3)...
customer 3 berhasil booking kamar #5 (tersisa 0 kamar)
customer 9 check-out dari kamar #1 (tersisa 1 kamar)
customer 1 check-out dari kamar #2 (tersisa 2 kamar)
customer 2 check-out dari kamar #3 (tersisa 3 kamar)
customer 8 check-out dari kamar #4 (tersisa 4 kamar)
customer 3 check-out dari kamar #5 (tersisa 5 kamar)

=== Semua customer selesai ===
```

Test Case yang dicoba :

- 10 pelanggan dengan 5 kamar
- Setiap pelanggan mencoba 3 kali bila gagal
- Durasi menginap 5 detik

Hasil :

Program berjalan sesuai ekspektasi. Tidak ada race condition pada output karena sudah disinkronisasi

4. ANALISIS HASIL

- Apakah sesuai ekspektasi?
Ya, setiap pelanggan berhasil bergantian memesan kamar tanpa konflik data.
- Masalah & Solusi
Awalnya output acak dan tumpang tindih, diselesaikan dengan `pthread_mutex_lock` disetiap print.
- Performa program
Program berjalan efisien, semua thread aktif bersamaan namun sinkron berkat semaphore.
- Pelajaran yang didapat
Pentingnya mekanisme sinkronisasi (Semaphore & Mutex) dalam mengelola thread concurrency.

KESIMPULAN

Berdasarkan hasil praktikum Synchronization System, dapat disimpulkan bahwa penerapan mekanisme sinkronisasi dengan menggunakan Semaphore dan Mutex berperan penting dalam menjaga kestabilan jalannya proses multithreading.

Melalui simulasi sistem pemesanan kamar hotel, setiap thread (pelanggan) dapat melakukan proses booking secara bergantian tanpa menimbulkan konflik data (race condition).

Sinkronisasi menjamin bahwa hanya satu thread yang mengakses sumber daya bersama pada satu waktu, sehingga hasil program tetap konsisten dan sesuai harapan.

Eksperimen tambahan dengan jumlah thread lebih banyak juga menunjukkan bahwa mekanisme sinkronisasi tetap berjalan dengan baik, menunjukkan keandalan konsep ini dalam sistem paralel yang kompleks.

1. Rangkuman Apa yang Dipelajari

Dari praktikum ini, mahasiswa mempelajari:

- Cara kerja sinkronisasi proses menggunakan Semaphore dan Mutex dalam pemrograman paralel.
- Dampak dari tidak adanya sinkronisasi yang menyebabkan race condition.
- Implementasi thread-safe programming untuk menjaga konsistensi data saat banyak proses berjalan bersamaan.
- Pentingnya sinkronisasi dalam sistem operasi, terutama dalam resource sharing dan process coordination.

2. Tantangan Yang Dihadapi

- Selama praktikum, tantangan yang dihadapi antara lain:
- Kesulitan memahami perbedaan antara Mutex (penguncian tunggal) dan Semaphore (pengaturan banyak izin).
- Kesalahan dalam urutan lock dan unlock yang menyebabkan deadlock.
- Penentuan jumlah thread dan resources yang optimal agar program berjalan stabil.
- Penanganan output agar tidak tumpang tindih saat banyak thread berjalan bersamaan.

3. Saran Pengembangan

- Untuk pengembangan berikutnya, disarankan agar:
- Menambahkan interface grafis (GUI) agar hasil simulasi lebih mudah dipahami secara visual.

- Menggunakan thread pool untuk efisiensi pemakaian sumber daya.
- Melakukan pengujian dengan beban yang lebih besar untuk melihat batas kemampuan sinkronisasi.
- Menambahkan logging system agar aktivitas setiap thread dapat direkam untuk analisis lebih lanjut.

REFENSI

Buku / Artikel yang Dihubungkan:

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2020). Operating System Concepts (10th ed.). Wiley.
2. Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson Education.
3. Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th ed.). Pearson.

Website yang Digunakan:

1. <https://www.geeksforgeeks.org/synchronization-in-operating-system/>
2. <https://www.tutorialspoint.com/process-synchronization-in-operating-system>
3. <https://www.programiz.com/c-programming/c-multithreading>